

Discord Bot

運用ステータスボード・復旧機能

技術者向け実装解説資料

観測・復旧・監査を一つの運用制御面として設計する

仕様書: Discord Bot 運用ステータスボード・復旧機能 実装仕様書 v1.0

対象: 保守担当者 / 運用担当者 / SRE / 実装レビュー担当者

作成日: 2026-08-01 | 実装検証: 127 tests passed

本資料は実装コードと仕様書を対応付けた設計解説です。運用開始時に必要な外部設定や実環境スモークテストも明示します。

目次と読み方

本資料は、機能の使い方ではなく、なぜこの実装になっているかを保守可能な粒度で説明する。最初に全体像を読み、その後状態モデル、更新制御、復旧、監査、運用手順の順で確認すると、障害発生時にどのデータを見てどの操作を選ぶべきかを追やすい。

章	内容
1	設計目標とスコープ
2	システム構成と責務分離
3	Operational Snapshotの設計
4	ステータスボード更新エンジン
5	復旧操作と安全制御
6	起動復元と障害分離
7	監査・ヘルス・可観測性
8	更新フックと整合性モデル
9	テスト戦略と検証結果
10	運用ランブック
11	保守時の判断基準と拡張指針
付録	コードマップと受入チェックリスト

コード参照はリポジトリ相対パスと行番号で記載する。実際のデプロイ環境では、Discord権限、MongoDB接続、コマンド登録の3つがコード外の前提条件になる。

1. 設計目標とスコープ

この実装のステータスボードは、単なる監視画面ではない。運用中の機能状態を共通形式で観測し、復旧可能な操作を提示し、操作結果を監査可能な状態にするための運用制御面である。したがって設計の中心はUIではなく、状態の読み取り境界と副作用の安全境界に置かれている。

1.1 解決する運用上の問題

- 複数のスケジューラ、VC監視、募集、告知、パネルが別々の状態を持ち、再起動後に全体像を把握しにくい。
- Discordへの投稿成功とMongoDBへの状態保存成功がずれると、再送や二重処理が起こり得る。
- 復旧操作が古い画面の状態を前提に実行されると、既に解決した問題を再処理したり、別の実行と衝突したりする。
- Botが付与した一時ロールと、人手で付与したロールを区別できないまま一括解除すると、運用事故になる。
- 障害が一つの機能に閉じず、別ギルドや別モジュールの処理まで停止する可能性がある。

1.2 設計原則

原則	実装上の意味	主な実装
観測と操作を分離	Snapshotは読み取り専用。復旧操作は管理サービスを経由し、最新状態と権限を再確認する。	operational-status-service.js / operational-management-service.js
副作用の前に所有権を確定	Mongo leaseと原子的claimで、同じギルド・同じ対象に対する同時実行を抑止する。	mongo-lease-lock-store.js / 各処理の claim
不確実な結果を成功扱いしない	Discord投稿後に永続化できない場合は未確認・部分成功として残し、無条件に再送しない。	Kokuchi / Otebo state machine
部分失敗を局所化	モジュール、ギルド、ボード更新の境界で失敗を捕捉し、他の観測と処理を続ける。	Promise.allSettled / per-module try-catch
人手のデータを保護	Bot所有フラグとsource identityが一致する記録だけを自動回収する。	VoiceParticipantRoleGrant

2. システム構成と責務分離

実装は、Discordのイベントハンドラにすべての判断を集約せず、状態を観測するサービス、ボードを書き込むサービス、復旧操作を実行するサービスに分離している。これにより、定時処理、ボタン操作、再起動復元のどの入口からでも同じ永続状態と排他規則を利用できる。

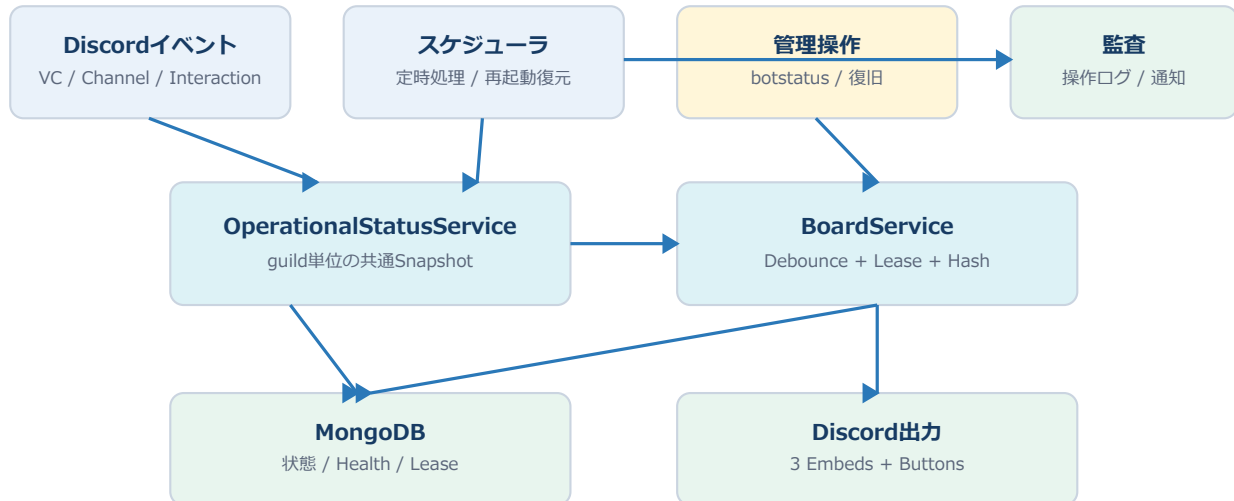


図2-1. DiscordイベントとスケジューラをSnapshotへ集約し、出力と復旧を別の副作用境界に分ける。

2.1 主要コンポーネント

コンポーネント	責務	設計上の境界
OperationalStatusService	ギルド単位でsystem, kokuchi, splitvc, recruitment, automation, panels, voiceを収集し、共通Snapshotを返す。	読み取り。各モジュールは独立して例外処理。
OperationalStatusBoardService	Snapshotを3 Embedへ変換し、既存メッセージを更新または再作成する。	Discord Message操作とboard leaseを担当。
OperationalManagementService	show, refresh, details, manage、復旧操作の権限・再確認・監査を担当する。	ユーザー操作とRecovery Serviceの仲介。
MongoLeaseLockStore	TTL付きの所有権をMongoDBへ保存し、同じキーの同時処理を抑止する。	プロセスをまたいだ排他。
OperationalHealthState	DB状態、Snapshot状態、起動復元結果、最後のエラーを保存する。	ヘルス表示と障害調査の基礎。
OperationalActionLog	操作主体、対象、前後Snapshot、結果、エラーをTTL付きで保存する。	監査のdurable側。

2.2 重要なデータフロー

- 状態変化は各処理のfinallyやイベント境界からrequestOperationalStatusRefreshへ通知される。
- ボード側は2秒のデバウンスで同一ギルドの要求をまとめ、最新Snapshotを1回取得する。
- 管理操作は実行前Snapshotで操作可否を再判定し、実行後Snapshotを取得して監査する。
- SnapshotのDB書き込みは観測結果を残すためのもので、観測失敗が機能本体を停止させない。

3. Operational Snapshotの設計

Snapshotは、管理者が最初に見るべき共通契約である。すべてのモジュールはkey, label, severity, summary, details, issues, blocking, availableActions, observedAtを持つ。表示用の短いsummaryと、管理操作・調査用のdetailsを分離している点が重要である。

3.1 共通形式

```
{
  guildId: "...",
  observedAt: "ISO-8601",
  attentionCount: 0,
  recommendationCount: 0,
  availableActions: [],
  modules: {
    system: {
      key, label, severity, summary, details,
      issues: [{ code, message, blocking }],
      blocking, availableActions, observedAt
    }
  }
}
```

3.2 モジュール別の観測対象

モジュール	観測する状態	blockingになる代表例
system	Discord ready、Mongo ping、起動復元、board最終更新時刻	Discord未ready、DB未接続
kokuchi	予約候補、投稿確認、集合VC権限復元、タイマー状態	復元保留、予約失敗、タイムアウト、孤立イベント
splitvc	実行中セッション、待機監視heartbeat、終了通知、ロール回収	stale session、failed session
recruitment	定時募集、Otebo状態、期限切れ、参加者数、cleanup	期限切れ、cleanup_pending
automation	Fukyo、Bump、ScheduledAction、失敗履歴	失敗した自動処理、設定不備
panels	DB記録とDiscordメッセージの存在確認	主要画面ではwarningまたはunknownとして提示
voice	一時ロール、退出予定、VC監視、集合VC復元	ロール回収失敗、監視stale、復元保留

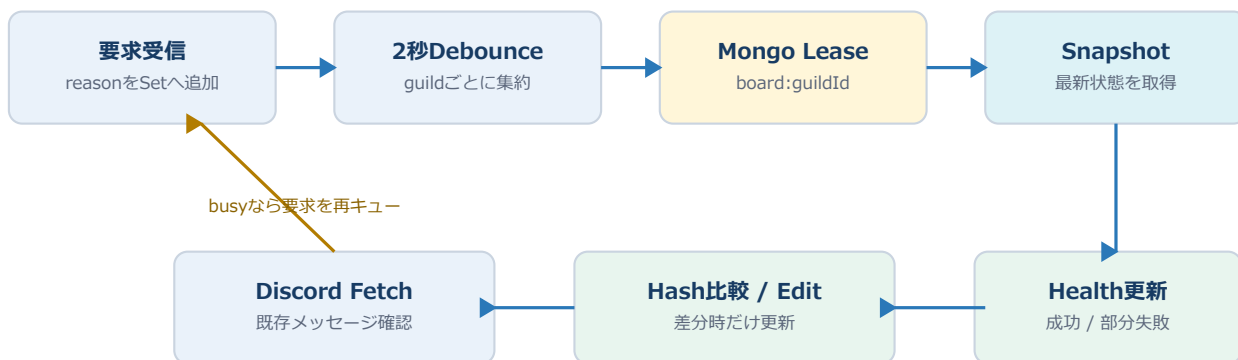
3.3 unknownとdisabledの使い分け

disabledは機能が意図的に未設定である状態を表す。一方、unknownは判断に必要な情報が読めない状態であり、healthyとして扱ってはならない。GuildSettingsの読み取りに失敗した場合、設定依存のkokuchi, recruitment, automation, panels, voiceはunknownへ上書きされる。splitvcのように設定依存性が異なるモジュールは独立して評価される。

設定読み取り失敗を空設定として扱わないことが、安全上の要点である。空設定として扱うと、実際には稼働中の処理を未設定・無効と誤認し、期限処理や復旧候補を見落とす。

4. ステータスボード更新エンジン

ステータスボードはギルドごとに1つのDiscordメッセージを所有し、状態変化時は同じメッセージを編集する。Message IDが消失した場合だけ再作成し、更新経路が異常終了してもボード障害がBot本体の機能障害へ波及しないようにしている。



更新中に届いた要求は完了後に最新Snapshotで再実行

図4-1. 更新要求はギルド単位で集約し、Mongo leaseとメッセージhashで副作用を抑える。

4.1 更新キューの状態機械

状態	意味	次の動作
pending	要求を受け、2秒のデバウンス待ち	同一guildのreasonとresolverを集約
running	Snapshot取得またはDiscord/Mongo処理中	到着した要求は次回バッチへ残す
busy	別のboard/configure/removeがlease保持中	要求を失わず、デバウンス後に再試行
completed	更新結果をresolverへ返却	保留要求があれば最新Snapshotで再実行
stopped	Bot終了時にタイマーを停止	保留要求をstoppedとして完了

4.2 hashと時刻表示の正規化

Payload hashはEmbedとComponentをJSON化して計算する。ただしDiscordの相対時刻表示はSnapshotごとに変化するため、を観測用ブレースホルダーへ正規化している。この正規化により、時刻が経過しただけの更新で毎回Discord APIを呼ばず、実質的な状態変化だけを編集対象にできる。

4.3 メッセージ消失と権限エラー

- Discord Unknown Message、code 10008、HTTP 404は、対象メッセージが消えたと判断して再作成する。
- その他のfetch失敗は不明な状態として記録し、重複送信を避けるため自動再作成しない。
- チャンネル不在・権限不足では、設定を別チャンネルへ自動移動しない。管理者が設定を見直す。
- ボード自身の失敗はOperationalHealthStateへ残し、Snapshotや他機能の処理は継続する。

4.4 起動時の復元順序

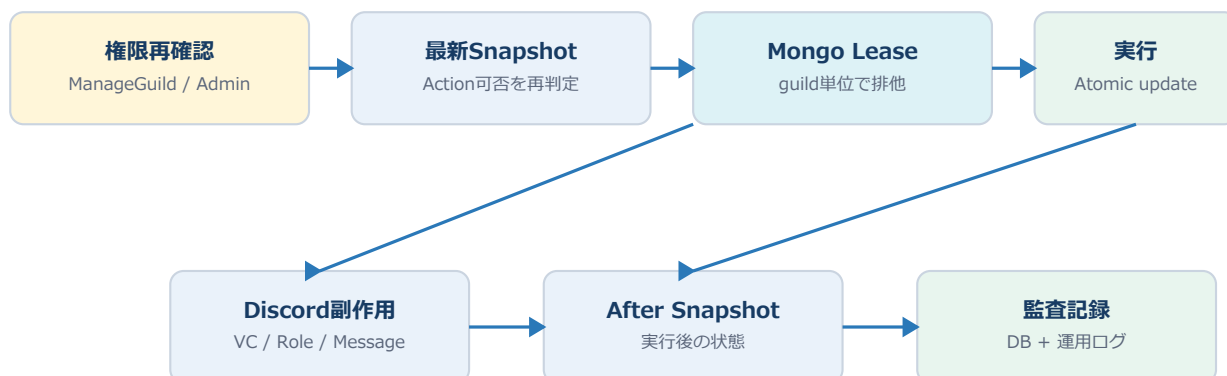
```

ClientReady
-> Promise.allSettled(all feature restorers)
-> persist startup restore health
-> start hourly board timer
-> restore configured status boards
-> initial call-wait processing

```

5. 復旧操作と安全制御

復旧は、管理者がボタンを押した時点の画面状態を信頼して実行するのではなく、実行直前に最新Snapshotと権限を再取得する。副作用の前後をSnapshotに残し、監査記録が完全に保存できない場合は成功を部分成功へ格下げする。



成功だけでなく、部分成功・監査失敗・復旧保留を状態として残す

図5-1. 権限再確認、最新状態、lease、実行、監査の順に境界を置く。

5.1 操作と権限

操作	必要権限	安全上の意図
show / refresh / details / manage	ManageGuild	運用状態と候補操作を閲覧できる管理者に限定
通常のkokuchiキャンセル	ManageGuild	通常復旧。最新候補を選んで実行
パネル再設置 / 期限切れ募集終了	ManageGuild	モジュール状態に候補がある場合だけ提示
参加者ロール一括解除	ManageGuild + Bot ManageRoles	Bot所有の記録だけを対象にする
kokuchi強制終了	Administrator	外部副作用を伴う強制操作のため強化
状態のみ解除	Administrator	権限を手動確認済みの場合に限定
確認ボタン再実行	クリック時に再確認	古いメニューや権限変更を無効化

5.2 Kokuchiの排他と復元保留

Kokuchiは、通常キャンセル、強制終了、状態のみ解除、予約キャンセル、集合VC権限復元で同じguild単位のkokuchi-recovery leaseを利用する。権限復元に失敗した場合、予約や状態を成功扱いで消去せず、gatheringVcRestorePendingを残す。これにより、Discord権限の復旧が未完了なのに新しい告知を開始する事故を防ぐ。

5.3 Bot所有ロールの保護

参加者ロールの回収は、設定された対象ロール、guild、member、sourceType、sourceId、grantedByBot=true、active/removing/failedという条件で記録を絞る。Discord上でロールを持っているという事実だけでは回収対象にしない。手動付与や外部Botによる付与を保護し、回収後は該当grantの状態だけをremovedへ更新する。

```

VoiceParticipantRoleGrant
  guildId + memberId + roleId
  sourceType + sourceId
  grantedByBot: true
  status: active | removing | failed

```

```
AUTO_CLEANUP = roleId AND grantedByBot=true AND source matches
```

5.4 Oteboの不確実な投稿と期限処理

- 投稿後の永続化に失敗した場合はpublished_unconfirmedとして残し、自動再送しない。
- deadlineはotebo-deadline:guildId:recruitmentId leaseで一意に処理する。
- success_processing、success_notified、cleanup_pending、failedは、ロール回収が未完了の可能性として可視化する。
- 期限切れ募集の終了と、成功後のロール回収を別の状態・別の処理として扱う。

6. 起動復元と障害分離

Botの再起動は通常運用の一部であるため、タイマーをメモリだけに置かず、MongoDBに状態と実行候補を残す。起動時は全復元処理をPromise.allSettledで走らせ、1機能の復元失敗が他ギルド・他モジュールの復元を止めないようにする。

6.1 復元対象の分類

分類	例	復元方針
永続スケジュール	Kokuchi reminder、Bump、Otebo deadline、ScheduledAction	保存されたexecuteAtを読み、claim後に再開
中断可能な処理	splitvc waiting monitor、VC監視	セッションとheartbeatを確認し、staleなら状態を可視化
外部投稿との不確実性	Kokuchi投稿、Otebo notice	published_unconfirmedを復元し、無条件再送をしない
Discordパネル	Profile / VC control panel	DB記録とメッセージ存在を照合し、必要時に再設置
一時ロール	participant role grant	Bot所有記録を読み、回収可能な状態だけを再処理

6.2 ヘルスエンドポイントの意味

GET /healthはDiscord ready、MongoDB ready、startupRestoreCompleted、startupRestoreFailed、shuttingDownを組み合わせる。起動復元中、復元失敗、DB切断、終了中はHTTP 503となる。単にプロセスが生きているかではなく、運用可能な制御面として準備できたかを返す。

```

ok = discordReady
    && mongoReady
    && startupRestoreCompleted
    && !startupRestoreFailed
    && !shuttingDown

```

6.3 障害分離の境界

失敗箇所	ユーザー／運用への表示	継続する処理
Snapshotの単一モジュール	該当module=unknown、issue=read_failed	他モジュールのSnapshotとボード更新
GuildSettings読み取り	依存module=unknown、systemにも設定エラー	設定非依存の観測と別ギルド
ボードDiscord操作	board health=partial、lastRefreshError保存	Bot本体のイベント・スケジューラ
監査DB保存	操作結果をpartialへ格下げ	副作用は巻き戻さず、再試行判断を促す
単一ギルドの起動復元	startup restore partial	他ギルドの復元、ヘルス提供

7. 監査・ヘルス・可観測性

運用操作は、画面に成功と表示しただけでは完了としない。OperationalActionLogへの永続保存と、設定された運用ログチャンネルへの通知を別々に確認し、どちらか一方しか成功しなかった場合はpartialとする。

7.1 OperationalActionLog

フィールド	内容	用途
guildId / actorUserId	対象ギルドと操作したユーザー	誰がどこで操作したか
actionType / targetId	復旧操作の種類と対象予約等	再現性のある対象特定
before / after	操作前後のSnapshotまたは対象状態	状態遷移の監査
result	success / partial / failed	監査保存状況を含む最終判定
errors	最大20件の短縮エラー	画面とログに出す要約
cleanupAt	30日後のTTL削除時刻	監査データの保持期間管理

7.2 成功判定の実装上の注意

```

before = snapshot()
result = executeAction() // action■■■■■Snapshot■■■■■
after = snapshot()
audit = durable DB log + operational channel log

auditStatus != recorded
-> errors■■■■
-> success■■■partial■■■■

```

7.3 ボードのヘルス情報

- lastSuccessfulRefreshAtは、hashが変わらず編集しなかった場合も成功更新として進める。
- lastRefreshErrorはチャンネル不在、権限不足、Message fetch失敗、Discord更新失敗を保存する。
- Snapshot側はboardの最終成功時刻を見て、2時間以上更新がない場合にstale warningを出す。
- ヘルス書き込み自体の失敗は、観測・復旧の主処理を失敗扱いにしない。ログに残して次回観測へ委ねる。

8. 更新フックと整合性モデル

ステータスボードはデータベースの変更通知を直接購読していない。そのため、状態を変更する処理の成功・失敗・finally境界から明示的に更新要求を送る。要求は結果の一部ではなく、状態が変わったことを知らせるヒントであり、最終的な正しさは次回Snapshotの読み取りで決まる。

フック群	代表イベント	設計意図
Discordイベント	VoiceStateUpdate、ChannelCreate/Delete/Update、InteractionCreate	人の操作やDiscord側の状態変化を反映
Kokuchi	予約、告知、事前通知、集合VC、キャンセル、復元	タイマーと復旧状態の遷移を反映
SplitVC	転送、待機VC、終了通知、ロール回収	長時間処理の途中経過を反映
Recruitment/Otebo	投稿、参加、期限、成功通知、cleanup	不確実投稿と回収保留を反映
Automation	Fukyo weekly、Bump reminder、ScheduledAction	バックグラウンド処理の終了を反映
Panels/Profile/VC	設置、再設置、削除、公開後更新	永続記録とDiscordメッセージの差分を反映

8.1 一貫性の強さ

対象	一貫性モデル	許容される中間状態
副作用の所有権	Mongo lease / atomic claim	lease期限切れ後の再取得
同一ギルドのボード	メモリキュー + Mongo lease	最大2秒の遅延、更新中の再実行待ち
Snapshot	読み取り時点のbest-effort集約	モジュール単位unknown / partial
Discordメッセージ	hash比較付きのupsert-like更新	一時的なfetch不可、Unknown Message再作成
監査	DB durable + チャンネル通知	片方だけ成功したpartial

この実装は、全処理を単一トランザクションに含む設計ではない。Discord APIとMongoDBをまたぐため、lease、claim、状態機械、再起動復元、partial表示を組み合わせることで実用的な一貫性を作っている。

9. テスト戦略と検証結果

テストは、純粋な日時計算だけでなく、失敗後に何を永続化し、再起動後に何を再実行し、手動ロールや不確実投稿をどう扱うかを中心に構成している。

検証領域	確認内容	結果
全体テスト	全テストスイートをnode --testで実行	127 tests / 127 pass / 0 fail
構文	package.jsonのlintスクリプトによる全対象node --check	pass
差分	git diff --check	pass。改行コード警告のみ
Snapshot	共通形式、6段階severity、設定読み取り失敗unknown	pass
Board	3 Embed、marker、管理ボタン、更新競合再実行	pass
Recovery	Kokuchi復旧、権限復元失敗、lease、監査降格	pass
Recruitment	Otebo投稿不確実、cleanup_pending、期限処理	pass
Role safety	Bot所有grantだけの回収、source identity保持	pass

9.1 競合再実行の回帰テスト

今回追加したテストでは、最初のDiscord message.editを意図的に停止し、その最中に2回目のrequestRefreshを送る。最初の編集が解放された後、2回目のSnapshotを読み直して再編集すること、両要求のPromiseが更新結果で完了することを確認する。

```

firstRequest = requestRefresh(guild, "first")
await firstEditStarted
secondRequest = requestRefresh(guild, "during-first-update")
snapshotVersion = 2
releaseFirstEdit()

assert firstRequest.status == "updated"
await secondEditStarted
assert editCount == 2
assert secondRequest.status == "updated"

```

9.2 この検証で保証していないもの

- 実際のDiscordサーバーでのBot権限、Message fetch、Embed編集、Interaction応答時間。
- 実MongoDB Atlasへの接続、インデックス作成、ネットワーク断、複数プロセス同時実行。
- 本番ホスティングのSIGTERM、再デプロイ、監視サービスによる/health判定。

10. 運用ランブック

10.1 起動前チェック

- Node.jsは22.12.0以上。package.jsonのenginesもこの条件へ統一済み。
- DISCORD_TOKEN、DISCORD_CLIENT_ID、MONGODB_URIが設定されている。値そのものはログへ出さない。
- Bot招待にbotとapplications.commandsのscopeがある。
- Board対象チャンネルにViewChannel、SendMessages、EmbedLinks、ReadMessageHistoryがある。
- 機能に応じてManageRoles、ManageChannels、MoveMembers、Connect、Set Voice Channel Statusを確認する。
- 通常メッセージ本文に反応する機能を使う場合、Message Content Intentを確認する。

10.2 初回起動とボード設置

```
npm install
npm run register
npm start

/setting status_board channel:#■■■■■■■■■
/botstatus show
/botstatus refresh
/botstatus manage
```

/setting status_boardは指定チャンネルにボードを作成し、DBへguildId、channelId、messageId、payloadHashを保存する。以後の状態変化は既存メッセージを編集する。チャンネル変更時は新メッセージの保存成功後に旧メッセージを削除する。

10.3 ヘルス確認

```
GET /
GET /health

■■■■:
HTTP 200
ok=true
discordReady=true
mongoReady=true
startupRestoreCompleted=true
shuttingDown=false
```

10.4 障害時の初動

画面の状態	最初に確認するもの	推奨対応
system=error / MongoDB disconnected	接続文字列、Atlas allowlist、Mongo health	復旧まで操作を増やさず、再接続と/healthを確認
board=warning / channel-missing	設定channelIdとDiscordチャンネル存在	/setting status_boardで再設定
board=warning / permissions	Botの4権限とチャンネルoverwrite	権限修正後に/botstatus refresh
kokuchi restore pending	集合VCの権限スナップショットと現状	restore_gathering_vcを実行。失敗時はforceを急がない
recruitment cleanup_pending	対象OteboとVoiceParticipantRoleGrant	Bot所有grantだけを確認し、remove_participant_rolesを実行
module unknown	issue code=read_failed/settings_unavailable	DBとGuildSettingsを復旧し、再度refresh

11 - 保守時の判断基準と拡張指針

11.1 新しいモジュールを追加する場合

- Snapshotの共通形式を満たすmoduleを追加する。summaryは短く、detailsにIDや時刻などの調査情報を置く。
- DB read、Discord read、設定 readを混ぜず、例外境界をモジュール内に置く。
- 障害時はhealthyではなくunknownまたはwarningへ遷移させる。
- 復旧可能な状態にはavailableActionsを追加し、管理サービス側でも最新Snapshotと権限を再確認する。
- 状態を変えるすべての成功・失敗・finally境界からrequestOperationalStatusRefreshを呼ぶ。
- 再起動後に必要なタイマーやclaim状態はMongoDBへ保存し、restoreに追加する。

11.2 新しい復旧操作を追加する場合

検討項目	必須条件
権限	閲覧系はManageGuild、破壊的・状態のみ操作はAdministratorを基準にする
再確認	クリック時にユーザー権限、guild、対象、availableActionsを再取得する
排除	guildまたは対象単位のMongo leaseと原子的な状態遷移を使う
監査	before/after、actor、target、result、errorsを保存し、通知失敗もpartialへ反映する
復旧失敗	未確認・保留・部分成功を永続化し、同じ副作用を無条件に再送しない
ボード	操作完了後に最新Snapshotで更新要求を出す

11.3 避けるべき実装

- Discord上のロール保有者を全走査し、Bot所有記録なしに解除する。
- Message fetchが失敗したとき、理由に関係なく新しいメッセージを送る。
- 設定読み取り失敗をnullや空設定としてhealthy判定する。
- 管理画面に表示された対象IDを、実行時に再取得せずそのまま処理する。
- Discord副作用成功後のMongo保存失敗を成功扱いし、再起動時に再送する。
- ボード更新失敗をBot全体のエラーとしてthrowし、他のスケジューラを止める。

付録A. コードマップ

ファイル	担当領域	読むべきポイント
src/operational-status-service.js	Snapshot集約	module分離、unknown、severity、health write
src/operational-status-board-service.js	ボード更新	hash、marker、lease、debounce、再実行
src/operational-management-service.js	管理UIと監査	権限、最新Snapshot、before/after、partial
src/mongo-lease-lock-store.js	永続lease	ownerId、leaseUntil、重複upsert競合
src/models/operational-status-board.js	ボード永続モデル	guildId unique、messageId、payloadHash
src/models/operational-health-state.js	ヘルス永続モデル	startup restore、last error、DB status
src/models/operational-action-log.js	監査永続モデル	before/after、TTL、result、errors
src/kokuchi-recovery-service.js	Kokuchi復旧	共通lease、権限復元、状態のみ解除
src/bot.js	イベントと復元統合	ClientReady、hooks、role ownership、Otebo
test/operational-status.test.js	運用機能の回帰テスト	Snapshot、board、recovery、競合再実行

付録B. 本番受入チェックリスト

確認	判定	備考
npm test	127/127 pass	コードベース検証
npm run lint	pass	構文検証
git diff --check	pass	改行コード警告のみ
Node.js	22.12.0以上	package.jsonと一致
MongoDB	接続・index作成	起動ログと/healthで確認
Discord login	ready	ClientReadyログ
Startup restore	completed=true	失敗時はHTTP 503
Board setup	messageId保存	/setting status_board
Manual refresh	既存message edit	/botstatus refresh
Permission denial	再確認して拒否	ManageGuild/Admin
Audit	DB + channel log	片方失敗はpartial
Shutdown	Discord/Mongo close	SIGTERM smoke test

最終的な運用開始判定は、コードテストの成功だけでなく、実Discordサーバー・実MongoDB・ホスティングの3環境で受入チェックリストを通過した時点とする。